SPELL: Synthesis of Programmatic Edits using LLMs

DANIEL RAMOS, Carnegie Mellon University, USA, INESC-ID, Portugal, and Instituto Superior Técnico, Universidade de Lisboa, Portugal
CATARINA GAMBOA, Carnegie Mellon University, USA, LASIGE, Portugal, and Faculdade de Ciências, Universidade de Lisboa, Portugal
INÊS LYNCE, INESC-ID, Portugal and Instituto Superior Técnico, Universidade de Lisboa, Portugal
VASCO MANQUINHO, INESC-ID, Portugal and Instituto Superior Técnico, Universidade de Lisboa, Portugal
RUBEN MARTINS, Carnegie Mellon University, USA
CLAIRE LE GOUES, Carnegie Mellon University, USA

Library migration is a common but error-prone task in software development. Developers may need to replace one library with another due to reasons like changing requirements or licensing changes. Migration typically entails updating and rewriting source code manually. While automated migration tools exist, most rely on mining examples from real-world projects that have already undergone similar migrations. However, these data are scarce, and collecting them for arbitrary pairs of libraries is difficult. Moreover, these migration tools often miss out on leveraging modern code transformation infrastructure.

In this paper, we present a new approach to automated API migration that sidesteps the limitations described above. Instead of relying on existing migration data or using LLMs directly for transformation, we use LLMs to extract migration examples. Next, we use an agent to generalize those examples to reusable transformation scripts in `PolyglotPiranha`, a modern code transformation tool. Our method distills latent migration knowledge from LLMs into structured, testable, and repeatable migration logic, without requiring preexisting corpora or manual engineering effort. Experimental results across `Python` libraries show that our system can generate diverse migration examples and synthesize transformation scripts that generalize to real-world codebases.

CCS Concepts: • **Software and its engineering** → **Software evolution**; **Software maintenance tools**; *Programming by example*; • **Computing methodologies** → Natural language processing.

Additional Key Words and Phrases: API migration, library migration, program synthesis, code transformation, large language models, LLM agents

1 Introduction

Libraries are the cornerstone of modern software development. They provide modular functionality through their application programming interfaces (APIs) [10], which expose functionality while abstracting implementation details. Library *migration* refers to the process of replacing one software library used in a client codebase with another. Such migrations are necessary to accommodate evolving software requirements, e.g., a need for new features, license changes, or more modern or better-supported APIs [5, 7, 12, 38, 53]. Replacing an original library's API calls with those of the target library is a largely manual [28] and error-prone [27] task.

Authors' Contact Information: Daniel Ramos, danielrr@cmu.edu, Carnegie Mellon University, Pittsburgh, USA and INESC-ID, Lisbon, Portugal and Instituto Superior Técnico, Universidade de Lisboa, Lisbon, Portugal; Catarina Gamboa, cgamboa@andrew.cmu.edu, Carnegie Mellon University, Pittsburgh, USA and LASIGE, Lisbon, Portugal and Faculdade de Ciências, Universidade de Lisboa, Lisbon, Portugal; Inês Lynce, ines.lynce@inesc-id.pt, INESC-ID, Lisbon, Portugal and Instituto Superior Técnico, Universidade de Lisboa, Lisbon, Portugal; Vasco Manquinho, vasco.manquinho@inesc-id.pt, INESC-ID, Lisbon, Portugal and Instituto Superior Técnico, Universidade de Lisboa, Lisbon, Portugal; Ruben Martins, rubenm@cs.cmu.edu, Carnegie Mellon University, Pittsburgh, USA; Claire Le Goues, clegoues@cs.cmu.edu, Carnegie Mellon University, Pittsburgh, USA.

(a) Before (using `cryptography.fernet`)

```python
import hashlib
from cryptography.fernet import Fernet
(...)
def encrypt_document(document: str, key: bytes) -> bytes:
    cipher = Fernet(key)
    encrypted = cipher.encrypt(document.encode())
    return encrypted
(...)
```

(b) After (using `pycryptodome`)

```python
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad
(...)
def encrypt_document(document: str, key: bytes) -> bytes:
    cipher = AES.new(key, AES.MODE_CBC)
    iv = cipher.iv
    padded_data = pad(document.encode(), AES.block_size)
    encrypted = iv + cipher.encrypt(padded_data)
    return encrypted
```

(c) Representative generated tests (shared by both implementations)

```python
def test_round_trip_encryption():
    key = ...  # key derived for each library
    plaintext = "Attack at dawn."
    encrypted = encrypt_document(plaintext, key)
    decrypted = decrypt_document(encrypted, key)
    assert decrypted == plaintext

def test_encrypts_to_bytes():
    key = ...
    result = encrypt_document("hello", key)
    assert isinstance(result, bytes)
```

(d) Synthesized `PolyglotPiranha` program expressing the transformation

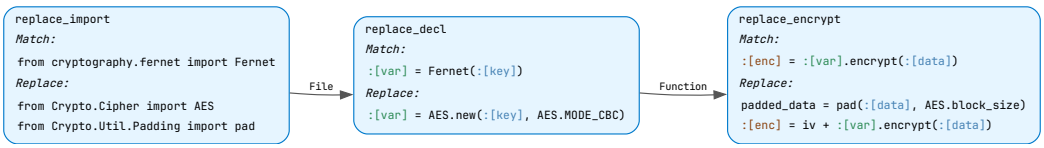


Fig. 1. Migration from `cryptography.fernet` to `pycryptodome`: (a) source implementation and (b) its migrated version; (c) representative generated tests shared by both implementations, validating behavioral equivalence; and (d) the `PolyglotPiranha` transformation script synthesized from this example.

The widespread and costly nature of library migrations motivates research into automation. Most existing methods mine software repositories that have already undergone migration [15–17, 26, 31, 35, 45, 54], generalizing and systematizing examples into reusable formats. However, these techniques are typically limited by their dependency on the availability and quality of historical migration data, which is scarce in practice [29].

Meanwhile, large language models (LLMs) have demonstrated potential and gained traction in assisting with code transformation, including in industrial settings (e.g., Amazon Q Transform [44]). However, directly using LLMs for migration is often unreliable, introducing unpredictable or formatting/stylistic changes or subtle semantic bugs [22]. They can also be inconsistent across similar migration patterns, and provide no reusable or inspectable artifacts for developers to understand what has been done. Finally, LLM-based migrations must inefficiently generate most changes from scratch. Crucially, even when LLMs succeed at individual file-level migrations, they offer no guarantee of *uniformity*: the same conceptual change may be realized differently across

files, complicating review, debugging, and rollback. In contrast, distilling LLM knowledge into deterministic transformation scripts ensures that every instance of a pattern is rewritten identically, a property that is essential for large-scale, auditable migrations in industrial codebases [25]. Moreover, the distilled scripts constitute reusable artifacts that can be inspected, version-controlled, and applied independently of any LLM, making them valuable beyond a single migration event—for example, as training data for future models or as documentation of API correspondences.

In this paper, we introduce SPELL (**S**ynthesis of **P**rogrammatic **E**dits using **LL**Ms), which, rather than using LLMs as black-box code translators, uses them as *knowledge sources* to systematically generate validated migration examples that are then distilled into precise, reusable transformation scripts. We frame API migration as an instance of the broader problem of synthesizing programmatic edits, i.e., deriving structured, repeatable code transformations from examples, where the edits happen to target library API replacements. Through pretraining on large-scale code corpora, LLMs learn joint representations of APIs and the semantic relationships between them, including cross-library mappings [43]. That is, LLMs implicitly encode rich knowledge about API relationships and usage patterns. This knowledge may be imperfect, or too noisy for direct application. However, our *key insight* is that it can be systematically extracted and validated to support migration script synthesis. This combines broad API knowledge encoded in LLMs with the reliability and efficiency of rule-based transformations, without requiring historical migration data.

A key benefit of SPELL is that it generates migration scripts in a domain-specific language (DSL) designed for code transformation (specifically `PolyglotPiranha` [25]). While most prior work produces or relies on custom transformation toolsets [15–17, 26], SPELL aims to leverage modern DSLs and frameworks like OpenRewrite [9], GritQL [47], and `PolyglotPiranha` [25] that support automated, consistent transformations across large codebases. SPELL's output thus serves as maintainable assets that can be inspected, version-controlled, debugged, and refined by developers. These DSLs are increasingly integrated into CI/CD pipelines at major technology companies like Uber [25] and Amazon [44], enabling our approach to naturally extend modern workflows. The overall use case can benefit both library maintainers supporting client codebases when pushing deprecating changes, and client teams migrating code without historical data available to mine.

SPELL takes a novel, hybrid approach to synthesizing migration scripts in the `PolyglotPiranha` language from its generated examples. SPELL first uses a classic approach for anti-unification-based synthesis. This step deterministically infers syntactic patterns from concrete code differences to produce an initial set of local transformation rules that express parts of a migration. However, `PolyglotPiranha`, like other transformation frameworks and DSLs [9, 32, 47], expresses migrations as combinations of locally-scoped rules, to capture semantic relationships, dependencies, chaining, or scoping. SPELL therefore combines classic synthesis with an agent workflow to refine, compose, and orchestrate the initially-inferred local rules into overall `PolyglotPiranha` programs that take full advantage of the expressive power of modern transformation DSLs.

Our results show that SPELL can synthesize correct, reusable migration scripts across diverse Python library migrations. Across ten migration tasks, it generated an average of 87 validated examples per task and inferred correct scripts for 61.6% in a single trial. SPELL outperformed MELT, a state-of-the-art prior approach, on every task. Moreover, the scripts generalized to real-world projects taken from open-source repositories. We kept our LLM budget deliberately low (< $100) for all experiments, demonstrating that the results are attainable cost-effectively, using small models. These findings show that structured migration knowledge can be distilled from LLMs and converted into reusable migration scripts.

In summary, we make the following contributions:

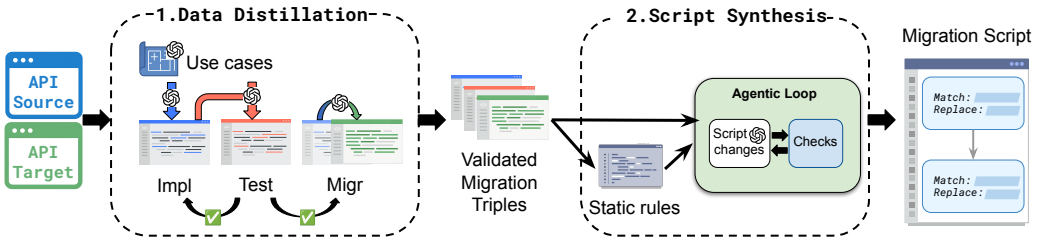


Fig. 2. Overview of Spell's pipeline. Given a pair of source and target APIs, Phase 1 distills data from LLMs in the form of use cases. These use cases guide the generation of examples that contain an implementation in the source API, a migration to the target API, and tests that validate both implementations. Phase 2 synthesizes the migration script using an initial set of statically inferred rules and an agentic loop to update them. The final output is a project-agnostic migration script that can be automatically applied to migrate from a source to a target API.

(1) A novel method for automatically generating reusable, validated migration scripts in an existing transformation DSL.
(2) A data generation approach for systematically extracting and validating structured migration knowledge from LLMs without requiring historical migration logs.
(3) A novel agentic workflow that leverages classic synthesis to transform examples into reusable migration scripts.
(4) A new dataset of 870 validated migration examples (an average of 87 per task) with corresponding test cases across ten `Python` migration tasks.
(5) An implementation, Spell, which achieves a 61.6% correctness rate in one-shot script synthesis and outperforms the previous state-of-the-art on every evaluated task. We demonstrate the application of these scripts to real, open-source projects.
(6) An open-source artifact including the Spell implementation, dataset, evaluation harness, and all synthesized scripts and results, publicly available [40].

2 Illustrative Example

Our pipeline consists of two phases: (1) data distillation, which generates code migration examples with tests, and (2) synthesis, where we abstract those pairs into reusable migration scripts. Figure 2 depicts the overall pipeline.

Data Distillation. The goal of data generation is to distill code migration knowledge from the LLM in the form of examples. To illustrate, consider the task of migrating between two Python cryptography libraries: `cryptography`[1] and `pycryptodome`.[2] Given a pair of libraries *(source, target)*, our approach uses an LLM to generate two implementations of the same functionality, one using each library. These paired implementations form the basis for migration examples that are later generalized.

The first step asks an LLM to generate *use cases*, i.e., tasks or features that could plausibly be implemented using either library. These use cases help focus the model on a shared functional goal that can be realized with both APIs. When asked to propose such a use case for `cryptography` and `pycryptodome`, `gpt4o-mini` responds:

[1]https://github.com/pyca/cryptography
[2]https://github.com/Legrandin/pycryptodome

> *A **File Encrypting Utility** can be implemented using `pycryptodome` or `cryptography` to allow users to encrypt and decrypt textual documents. High-level behaviour and API: (...)*

We then prompt the model to generate an *implementation* for this use case using the source library. Figure 1a shows one such implementation generated by the LLM using the `cryptography` library. We also prompt the LLM to generate *tests* for each implementation that verify functionality, and filter out clearly-spurious examples.

Finally, we prompt the LLM to *migrate* each implementation in `cryptography` to the alternative library `pycryptodome`. Figure 1b shows a migration generated for the code in Figure 1a. The migration prompt instructs the LLM to modify only the library-specific API calls, leaving the rest of the code unchanged. This enables the reuse of the tests generated for the original implementation on the migrated version. The tests provide a signal of migration success when the migrated code's tested behavior is the same as the original, and thus serve as a filter on the examples.

The process produces a set of code migration examples consisting of pairs of implementations of a use case, with shared test files. Section 3 describes data generation in detail.

Synthesizing Migration Scripts. The second phase abstracts synthetic migration examples into reusable migration programs that capture structural transformation patterns and can be applied across similar tasks or large codebases. We generate programs in the `PolyglotPiranha` [25] language, a DSL for source-to-source transformations. A `PolyglotPiranha` program is composed of `rules`, each with a `match` clause that identifies a code pattern and a `replace` clause that rewrites it. Template variables (e.g., `:[var]`) allow rules to generalize over specific names and expressions, acting like regular expressions, but for syntax-aware code patterns. Rules can be connected through labeled edges that specify how follow-up transformations should be triggered. This enables cascading and context-sensitive rewrites.

Figure 1d shows a `PolyglotPiranha` program synthesized from the example. The first rule, `replace import`, replaces the original import with one from the target library. This triggers the second rule within the same file, `replace decl`, which rewrites the object construction. The third rule, `replace encrypt`, updates how the encryption API is used. Template variables `:[var]` and `:[data]` allow the rules to generalize to different programs that share the same structure, but use different variable names or concrete expressions.

The output of this step is a set of interpretable migration programs that capture the essence of the transformation and can be adapted or extended to similar scenarios. Section 4 details script synthesis from synthetic examples.

3 Migration Data Distillation

Throughout this paper, we use "library" to refer to a software package (e.g., `requests`) and "API" to refer to the set of public methods and classes it exposes. A migration task targets a pair of libraries; the concrete changes involve replacing API calls from the source library with equivalent calls from the target library.

Figure 3 overviews the data distillation process, which extracts and validates LLM's pre-trained knowledge about migrating between two libraries. The process generates sets of equivalent programs using source and target libraries, with test cases that provide evidence of their functional equivalence. This data is the input for migration script synthesis (Section 4).

More precisely: given source library $\mathcal{S}$ and target library $\mathcal{T}$, we sample migration knowledge from an LLM as raw migration triples $\widetilde{\mathcal{M}} = \{(\tilde{s}_i, \tilde{t}_i, \tilde{m}_i)\}_{i=1}^{n}$. These triples are automatically generated and may contain redundancy, inconsistencies, or errors. We validate and filter this set to produce a high-quality subset $\mathcal{M} = \{(s_i, t_i, m_i)\}_{i=1}^{m}$, where $m \leq n$, and:

- s_i is an implementation using $\mathcal{S}$

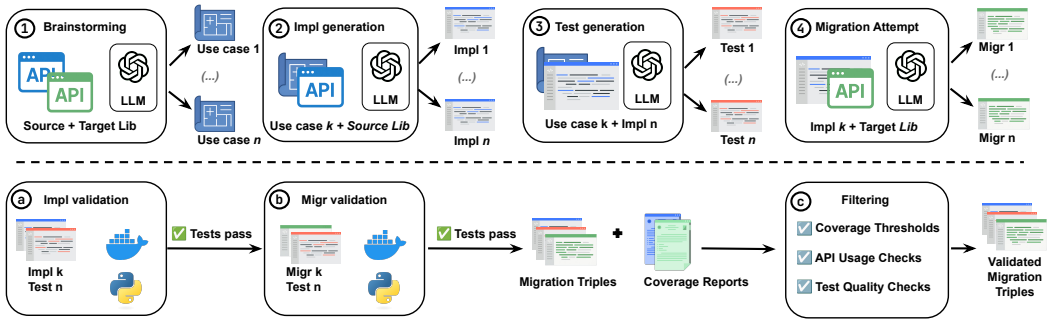


Fig. 3. Pipeline for generating migration examples using LLMs. Given a source–target library pair, we prompt the LLM to propose abstract *use cases* leveraging latent knowledge of both libraries (Step 1). For each use case, we generate multiple source library implementations (Step 2), and test suites (Step 3), and migrate them to the target library (Step 4). Validation entails implementation validation (a), migration validation (b), and quality filtering (c). The output is a collection of validated migration triples (implementation, test, migration).

- m_i is a functionally equivalent implementation using $\mathcal{T}$
- t_i is a test suite that executes and passes on both s_i and m_i

We use $\sim$ to denote sampled triples that may or may not be valid; notation without a $\sim$ refers to data/triples that have been validated. We now explain knowledge extraction (Section 3.1) and validation (Section 3.2). All prompts used in each stage of the pipeline are included in our publicly available artifact [40], including the system prompt for agentic synthesis, the use case generation prompt, and all implementation/test/migration prompts.

3.1 Model Sampling

The goal in model sampling is to generate a diverse initial set of example implementations of shared functionality, using $\mathcal{S}$ and $\mathcal{T}$. A nontrivial library can usually be used in a variety of ways. Thus, our process entails a multi-stage pipeline that first generates multiple *use cases* of functionality that can be implemented using $\mathcal{S}$ (Section 3.1.1), and separately generating *implementations* of those use cases using $\mathcal{S}$ (Section 3.1.2). For each implementation of each use case, the pipeline generates tests (Section 3.1.3), and finally migrates the implementation to use $\mathcal{T}$ (Section 3.1.4).

3.1.1 Use Case Generation. Given libraries $(\mathcal{S}, \mathcal{T})$, SPELL first prompts an LLM to generate multiple *use cases* C (Figure 3, Step 1). Each use case $C_j \in C$ represents a use case, program, or otherwise useful functionality that could be implemented using either library.

Generating use cases first decouples the creative task of brainstorming migration scenarios from the technical task of writing correct code. This serves to encourage a model to explore a wide space of API usages, beyond common boilerplate. It also provides a clear, shared semantic goal, encouraging consistency between source and target implementations. Finally, it allows the pipeline to retry implementation generation for promising use cases as necessary, without having to re-explore the entire conceptual space. Decoupling also allows the model to focus on brainstorming use cases that are not complex or otherwise unsuitable, before investing computational resources in their implementation, next.

Use case generation begins by using the model to create a small set of seed use cases that are then used for a self-instruct loop to generate additional migration scenarios. Self-instruction, in which a model produces its own task demonstrations using seed data, has proven effective in bootstrapping a model's existing knowledge in a variety of contexts [52]. We first generate p seed use cases by

prompting the model directly with a basic instruction. Each use case describes a use case involving a source library, along with a high-level summary of its functionality. These seed use cases are collected into a list and used as few-shot examples in subsequent prompts to generate more.

The use case generation loop proceeds by randomly sampling k prior use cases (seed or generated) to include in a new prompt, and asking the model to propose one new use case. This repeats until it produces a target number of use cases, where each use case demonstrates a way the source and target libraries can be used.

3.1.2 Implementation Generation. For each use case $C_j \in \mathcal{C}$, we prompt the model to generate a set of implementations $\widetilde{S}_j = \{\tilde{s}_{j,1}, \tilde{s}_{j,2}, ..., \tilde{s}_{j,n}\}$ using source library $\mathcal{S}$ (Figure 3, Step 2). Ideally, these implementations are modular, with a well-defined API. This simplifies migration to $\mathcal{T}$ while preserving the high-level interface, necessary to use the same tests on both.

The prompt thus instructs the model to implement the use case in a self-contained `Python` file, subject to the following constraints: (1) implement the API for use case C_j as a set of functions, (2) focus on API implementation; a small `main` block is optional, and (3) avoid external dependencies beyond the specified library $\mathcal{S}$.

We construct a multi-turn chat prompt that includes the original use case and implementation instructions. To obtain diverse outputs and increase the chances that the model produces usable implementations, we sample multiple completions using stochastic decoding.

The output of this step is a set of up to n potential *implementations* of each $C_j \in \mathcal{C}$.

3.1.3 Test Generation. For each use case-implementation pair $(C_j, \tilde{s}_{j,k})$, we generate p test suites $\widetilde{T}_{j,k} = \{\tilde{t}_{j,k,1}, \tilde{t}_{j,k,2}, ..., \tilde{t}_{j,k,p}\}$ (Figure 3, Step 3). Each test suite $\tilde{t}_{j,k,\ell}$ is intended to partially verify, and build confidence in the correctness of implementation $\tilde{s}_{j,k}$ (and, later, the migrated implementation using target library $\mathcal{T}$).

The ideal tests target an implementation's public API and core functionality, rather than internal implementation details that may not transfer. For example, tests for a program that encrypts files should not inspect the encrypted output directly, but should instead verify that decrypting a file restores the original content (among other functionality). This design encourages tests that capture *functional properties* as behavioral invariants such as round-trip consistency (e.g., `decrypt(encrypt(x)) == x`), rather than implementation-specific details. When the LLM generates such property-oriented tests, they serve as strong evidence that a migration preserves the semantic contract of the original code. We note that the extent to which generated tests capture functional properties depends on the model's test generation capabilities; we characterize this further in Section 5.6. The prompt instructs the model to generate unit tests (or end-to-end tests, if unit tests are infeasible) that validate the overall system behavior.

The prompt includes C_j, $\tilde{s}_{j,k}$, and the test generation instructions. We sample multiple test files per implementation to mitigate model hallucination as well as challenges in test generation generally, such as incorrect assertions. Sampling increases the chances of obtaining at least one valid test file for an otherwise correct implementation. Since our implementation is `Python`-specific, the prompt instructs the model to use either the `pytest` testing framework or property-based testing with `hypothesis`, when appropriate. This assumption could be easily ported to other languages.

The output of this step is multiple test files per implementation.

3.1.4 Migration. For each use case-implementation pair $(C_j, \tilde{s}_{j,k})$, we prompt the model to generate multiple migrated implementations $\widetilde{M}_{j,k} = \{\tilde{m}_{j,k,1}, \tilde{m}_{j,k,2}, ..., \tilde{m}_{j,k,q}\}$ in target library $\mathcal{T}$ (Figure 3, Step 4). Each $\tilde{m}_{j,k,r}$ represents an attempt to migrate $\tilde{s}_{j,k}$ while preserving its public API. This entails a multi-turn chat prompt that includes the original use case C_j and implementation $\tilde{s}_{j,k}$, and instructions asking the model to migrate the source implementation to use the target library. The

prompt further instructs the model to keep the same structure and function signatures, and only update the internal logic and API calls to use the target library.

The output of this step is the set of raw migration triples $\widetilde{\mathcal{M}} = \bigcup_{j,k}\{\tilde{s}_{j,k}\} \times \widetilde{T}_{j,k} \times \widetilde{M}_{j,k}$, where the union is taken over all use cases $C_j \in C$ and their implementations. $\widetilde{\mathcal{M}}$ may include incorrect or only partially migrated examples, that are validated and filtered as described in Section 3.2.

3.2 Validation

Not all sampled migration examples are valid. Initial or migrated implementations may be spurious or incomplete, tests may not run, etc. Thus, the validation step filters $\widetilde{\mathcal{M}}$ to produce a set of *valid* migration triples $\mathcal{M} = \{(s_i, t_i, m_i)\}_{i=1}^{m}$. In a valid triple, both source and migrated implementations expose desired APIs, pass the same tests, and are exercised with sufficient test coverage.

Implementation Validation. First, we validate initial use case implementations. For each $(\tilde{s}_{j,k}, \tilde{t}_{j,k,\ell}) \in \widetilde{S}_j \times \widetilde{T}_{j,k}$, we execute tests $\widetilde{T}_{j,k}$ against $\widetilde{s}_{j,k}$, retaining pairs where all tests pass. Recall that we generate multiple potential test suites per implementation (Section 3.1.3); this step can therefore produce multiple pairs containing a given $s_{j,k}$. We run each test file $\tilde{t}_{j,k,\ell}$ independently using `pytest` inside a `docker` container that includes a pinned `Python` version and a large set of `Python` libraries, including $\mathcal{S}$ and its dependencies.

Migration Validation. For each valid $(s_{j,k}, t_{j,k,\ell})$ pair and associated migration attempt $\tilde{m}_{j,k,r} \in \widetilde{M}_{j,k}$, we improve confidence in equivalence by confirming that $\tilde{m}_{j,k,r}$ also passes tests $t_{j,k,\ell}$ and dropping migration attempts whose tests fail, i.e., removing the entire candidate triple $(s_{j,k}, t_{j,k,\ell}, \tilde{m}_{j,k,r})$ from further consideration. Because the migrated code is expected to preserve the original API, the same test suite is expected to apply (migration attempts that break the shared tests are discarded). Since validation requires that both the source and migrated implementations execute and pass their shared test suite, all retained code is necessarily runnable.

Test-based Filtering. Finally, we also collect line-level coverage metrics for each run to build confidence that the test suites meaningfully exercise core logic. We simultaneously collect information on API usage for later analysis. We retain triples where shared tests achieve at least 60% line coverage in both implementations.

To construct the final validated $\mathcal{M}$: for each implementation $\tilde{s}_{j,k}$ of use case C_j part of more than one valid triple, we select the one with the highest post-migration test coverage. We include implementations part of only a single valid triple directly. These selected triples become $(s_i, t_i, m_i) \in \mathcal{M}$. Thus, each validated implementation contributes at most one triple to $\mathcal{M}$, ensuring high-quality examples while maintaining use case implementation diversity.

4 Migration Script Synthesis

Spell converts migration triples $\mathcal{M} = \{(s_i, t_i, m_i)\}_{i=1}^{m}$ (Section 3) into executable rewrite scripts in `PolyglotPiranha`, a state-of-the-art DSL and toolset for code transformation (Section 4.1). Our novel approach for script inference combines classical anti-unification with an agentic LLMs strategy. This hybrid is motivated by the structure of `PolyglotPiranha` programs: local rewrite rules (graph nodes) connected by a high-level strategy (labeled edges). Classical anti-unification excels at inferring precise local rules from concrete examples, but cannot infer the orchestration strategy, scope, or cascading effects. We therefore complement the traditional synthesis with an LLM-based technique to organize mechanically-inferred atomic rules into a coherent transformation strategy.

For each migration triple $(s, t, m) \in \mathcal{M}$ script synthesis entails: (1) **Rule Inference**: Automatically generate initial low-level transformation rules from source-target diff hunks (Section 4.2), and (2)

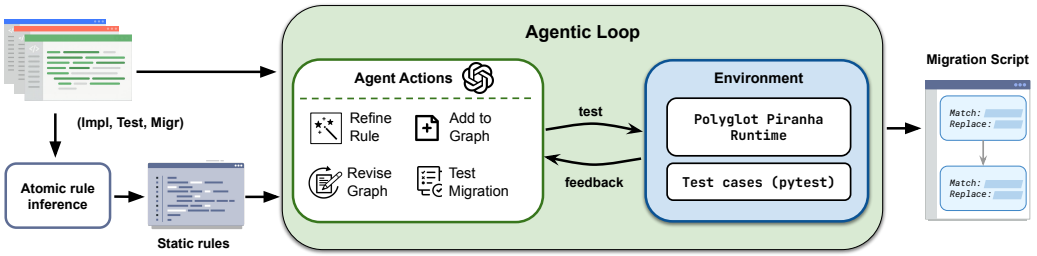


Fig. 4. Agentic workflow of SPELL's approach to migration script synthesis. Our pipeline leverages an atomic rule inference algorithm for creating an initial set of rules, which is then fed to an LLM agent for `Polyglot-Piranha` script generation. The agent iteratively applies changes to the script and gets feedback from the tests and `PolyglotPiranha`'s runtime to refine the rules.

Agent-based Orchestration: Use LLMs to refine and orchestrate the inferred rules into a rule graph (Section 4.3). This overall produces a migration program per successfully-processed triple in $\mathcal{M}$.

4.1 `PolyglotPiranha`

We infer migration scripts in `PolyglotPiranha`, a domain-specific language for source-to-source transformation developed and deployed at Uber [25]. `PolyglotPiranha` exemplifies a growing class of modern transformation DSLs [4, 9, 47] designed to support industrial-scale automated refactoring. These tools offer a structured, declarative alternative to ad hoc migration scripts.

To expand on key language constructs introduced in Section 2: `PolyglotPiranha` programs consist of local syntactic rewrite rules organized into a directed graph encoding application order, scoping constraints, and inter-rule dependencies. Each node defines a rule with a match and an optional rewrite clause; edges model control flow using scoped labels. I.e., an edge $\mathcal{R}_1 \xrightarrow{\text{Function}} \mathcal{R}_2$ reads as, "apply rule $\mathcal{R}_1$ and then apply rule $\mathcal{R}_2$ within the enclosing function where $\mathcal{R}_1$ was applied". Rules themselves are written as concrete syntax patterns (that closely resemble the target language's surface syntax). The concrete patterns are composed of strings and abstractions that match directly against concrete code. These abstractions are expressed via template variables (e.g., `:[x]`), which bind to concrete sub-expressions. For example, the rule `foo(:[args+])` ↦ `bar(:[args])` rewrites `foo(1,2,3)` as `bar(1,2,3)`, where `:[args]` binds to `(1,2,3)` during the transformation.

`PolyglotPiranha` programs run as depth-first traversal of the rule graph. The runtime begins with a queue of seed global rules and applies them depth-first until reaching a fixpoint. We refer interested readers to associated materials and documentation for full syntax and semantics [19, 25].

Thus, for each migration triple $(s, t, m) \in \mathcal{M}$, we aim to synthesize a `PolyglotPiranha` script $\mathcal{P} = (\mathcal{R}, \mathcal{E})$ where:

- $\mathcal{R} = \{r_1, r_2, ..., r_k\}$ is a set of transformation rules
- $\mathcal{E} \subseteq \mathcal{R} \times \mathcal{L} \times \mathcal{R}$ is a set of directed edges with labels from $\mathcal{L} = \{\text{File}, \text{Function}, \text{Class}, ...\}$

Each rule $r_i \in \mathcal{R}$ is a tuple (p_i, q_i) where p_i is a match pattern, and q_i is a replacement pattern. An edge $(r_i, \ell, r_j) \in \mathcal{E}$ indicates that after applying rule r_i, rule r_j should be applied within scope ℓ.

4.2 Atomic Rule Inference

Let $\Delta(s, m)$ denote the set of diff hunks between source implementation s and migration m. For each *diff hunk* $h \in \Delta(s, m)$, let h^- and h^+ denote the deleted and added lines, respectively.

We use an anti-unification algorithm to abstract these hunks into reusable rewrite rules. Anti-unification computes the most specific generalization of two expressions, producing a pattern that captures their common structure while abstracting over differences with placeholders. This makes it ideal for synthesizing migration rules that preserve the underlying structure yet allow for generalization. We adapt an existing anti-unification algorithm $\mathcal{A}$ (from Melt [41]) to infer an initial rule set:

$$\mathcal{R}_0 = \{r : r = \mathcal{A}(h^-, h^+) \text{ for } h \in \Delta(s, m) \text{ where } h^- \neq \emptyset\}.$$

Each rule abstracts shared elements in the *before* and *after* hunk.

For example, given the following hunk from a larger migration of `cryptography` to `pycryptodome`:

```
- encrypted_data = fernet.encrypt(data)
+ padded_data = pad(data, AES.block_size)
+ encrypted_data = iv + fernet.encrypt(padded_data)
```

Anti-unification, including heuristics to avoid overgeneralizing, produces: `:[x1]= :[x2].encrypt(:[x3])`

```
    ↦ padded_data = pad(:[x3], AES.block_size)
      :[x1]= iv + :[x2].encrypt(padded_data)
```

Note that the algorithm we adapt does not support rule generation for hunks in which code is only added without deletion [41]; we discuss this limitation in Section 6.

A further subtlety is that our diff-based representation uses hunks containing only modified lines, omitting unchanged context. Consequently, a single logical change—such as updating an import at the top of a file and a corresponding API call in a function body—may be split into separate hunks. Each hunk yields an independent atomic rule; the agentic orchestration step (Section 4.3) is responsible for composing these rules via scoped edges (e.g., applying an import rule and then a usage rule within the same file), thereby recovering the logical unity of such fragmented changes.

4.3 Script synthesis

Rule inference provides initial ruleset $\mathcal{R}_0$, but no composition strategy. Agentic script synthesis constructs the full script $\mathcal{P} = (\mathcal{R}, \mathcal{E})$ through iterative refinement. This refinement also allows Spell to correct initial rules r that overgeneralize or are incorrect. We use the term *agentic* to describe a workflow in which an LLM autonomously selects actions from a defined action space, executes them via external tools (here: the `PolyglotPiranha` interpreter and a containerized test runner), observes structured feedback, and adapts its strategy accordingly. This goes beyond repeated prompting: the model performs *planning* (choosing which rule to refine or whether to restructure the graph), *tool use* (invoking `PolyglotPiranha` and pytest as external tools), and *self-correction* (interpreting runtime errors or test failures to revise its approach). The distinction is analogous to that between a ReAct-style agent [55] and a simple chain of LLM calls.

Figure 4 illustrates the workflow. It takes as input a migration triple $(s, t, m) \in \mathcal{M}$ and the initial rule set $\mathcal{R}_0$ inferred from diff $\Delta(s, m)$ between the source and target implementations. The model's task is to develop the overarching strategy comprising the full script while disambiguating conflicting rules and deciding on scopes and application order; and improving the transformation rules as necessary by, for example, resolving naming problems. The rest of this Section details key components of the approach.

System prompt. We assume a model is unfamiliar with the relatively new `PolyglotPiranha` language. The system prompt explains language syntax, semantics, and runtime behavior, via: (1) examples of simple rules, explanations of concrete syntax, and guidance on matching/rewriting; (2)

an explanation of rule graph, rule application strategies, scoping, and constructs for propagating template variables across rules; (3) examples of source code before and after migration, with detailed explanations of how `PolyglotPiranha` performs migration, and runtime traces, and finally (4) other guidelines, common pitfalls, and best practices.

Agentic Loop. For each migration triple $(s, t, m) \in \mathcal{M}$ and initial ruleset, SPELL's agentic loop iteratively takes actions and observes their effects until it produces a `PolyglotPiranha` program that transforms the source s_i into a migration $\hat{m}_i$ that passes the original test suite t_i, or until it completes 10 iterations.

The first iteration takes a partially-complete task template containing the migration triple and initial SPELL-inferred rules. We prompt the model to generate a `PolyglotPiranha` program that transforms s_i into m_i, following system guidelines. The prompt specifies that the produced program need not produce an exact token-level match with m_i, but should capture the intended semantic transformation. We (and the agent) verify correctness by running code produced by the eventual program on test suite t_i.

Each iteration, the model selects an action and executes it in a controlled environment, with result returned as feedback:

- **Refine / Create an Atomic Rule.** The model can create a new match-replace rule, or refine an existing one, and apply it to s_i in isolation to observe an atomic rule's behavior independently of other transformations.
- **Add a Rule to the Graph.** The model can add one or more new rules to the (initially empty) graph. The action specifies rule order and scope.
- **Revise the Current Rule Graph.** A model can revise the graph beyond atomic rule refinement by modifying portions of the graph, or regenerating it, enabling high-level restructuring of the transformation strategy.
- **Test Migration.** The model can use tests t_i to validate the current rule graph by applying it to s_i and running tests t_i against the result. If the tests pass, the loop terminates.

Environment. SPELL executes actions within a structured environment and provides results back to the model to guide the next agentic step. For rule refinement and integration, and rule graph revision, SPELL applies generated P to s_i using the `PolyglotPiranha` interpreter and returns the resulting migrated program as feedback. SPELL provides additional feedback when available, e.g., if applying P fails due to a syntax or `PolyglotPiranha` runtime error, SPELL returns the error message with a set of likely causes and correction suggestions. These are derived from common failure modes and known pitfalls in the language engine, akin to compiler diagnostic messages. If the script runs but does not modify s_i, SPELL offers possible explanations and common pitfalls, based on our experience with the tool. For testing, SPELL applies the script and runs the transformed code inside a containerized test environment. If all tests pass, SPELL records the migration as successful; Otherwise, it returns the test failures and error messages for the next iteration.

How the LLM acquires `PolyglotPiranha` knowledge. The `PolyglotPiranha` language is relatively new and unlikely to appear extensively in pretraining data. We address this by providing a detailed system prompt (≈4,000 tokens) that teaches the language from scratch: syntax, semantics, runtime behavior, common pitfalls, and worked examples with execution traces. The iterative feedback loop compensates for residual gaps: when the model produces a syntactically invalid rule or one that fails to match, the environment returns diagnostic messages (analogous to compiler errors) that guide correction.

Cross-file migrations. `PolyglotPiranha` supports cross-file transformations: a rule graph can propagate changes across files within a project. For example, updating an import in one file can

trigger dependent rewrites in other files that reference the same module. This capability is important for migrations that require coordinated changes, such as renaming a shared utility or updating a library's initialization pattern that is referenced across multiple modules.

Once we have the migration scripts, we can use them to update any codebase from the source to the target library, as the rules are project-agnostic. The developer responsible for the migration can review these concise rules to understand the necessary changes and then use `PolyglotPiranha` to automatically apply them.

5 Evaluation

We answer four research questions:

RQ1. (Script synthesis success rate): How effectively does Spell synthesize `PolyglotPiranha` migration scripts? We evaluate the extent to which the synthesis pipeline produces working `PolyglotPiranha` scripts from the validated migration triples.

RQ2. (Real-world applicability): How useful are Spell's scripts for migrating real-world client code? We test whether the Spell-produced `PolyglotPiranha` scripts, learned from synthetic examples, successfully migrate real open-source projects, to evaluate practical utility.

RQ3. (Comparison with prior work): How does Spell compare to Melt, a prior tool for automated refactoring, and to direct LLM-based migration? We compare Spell to Melt, the most-closely-comparable prior approach for migration rule inference in Python. We also compare against directly prompting an LLM to perform the migration, to evaluate the added value of distilling reusable transformation scripts over one-shot LLM generation.

RQ4. (Data quality): What is the quality of the synthetic migration examples from the data generation pipeline? A core novelty of Spell is its bootstrapping via synthetic examples. We assess the extent to which these examples are valid and diverse, and provide sufficient coverage of both source and target APIs and implementations.

5.1 Experimental Setup

Metrics. We evaluate Spell using the following metrics:

- **Success rate:** The percentage of validated migration triples for which Spell synthesizes a `PolyglotPiranha` script that, when applied to the source implementation s, produces output that passes the shared test suite t. A script is considered *successful* if it removes all source-library API calls (i.e., $\text{APIs}(\mathcal{P}(s), \mathcal{S}) = \emptyset$) and the transformed code passes the tests ($\mathcal{P}(s) \equiv_t m$). We allow at most 10 agent iterations per synthesis attempt; an attempt that does not meet both conditions within this budget is unsuccessful.
- **Sibling success:** Given a script $\mathcal{P}_{j,k}$ synthesized from one implementation of use case C_j, we test whether it also successfully migrates *other* implementations of the *same* use case C_j (i.e., sibling implementations $\{s_{j,i} : i \neq k\}$ generated for the same use case but via different LLM samples). This measures the generalizability of a script beyond the single example it was derived from.
- **API diversity:** The number of distinct API methods from both source and target libraries that appear across all validated migration triples, computed using the Jedi static analyzer [18].
- **Test coverage:** Line-level code coverage of the generated test suites on both source and migrated implementations, measuring how thoroughly the tests exercise the code.

Implementation. We implement Spell in `Python`. Both synthetic data generation and agentic synthesis are model-agnostic and use the OpenAI Chat Completion API [37] (adopted by most major LLM API providers). For data generation, we use the cost-effective `gpt-4o-mini` (a small

Table 1. Overview of migration tasks with synthesis results. Valid Triples: generated migration triples with >60% coverage. Success: percentage of migration triples for which each tool successfully generated a validated `PolyglotPiranha` script. Sibling Success: sibling implementations that at least one synthesized `PolyglotPiranha` script migrates successfully out of the total number of potentially-migratable sibling implementations.

Migration Task				Valid	Success (%)		Sibling
Source	Target	Domain	Stars (S / T)	Triples	SPELL	MELT	Success
`argparse`	`click`	CLI	N/A / 16.4k	215	44.2	17.2	91/170
`jinja2`	`mako`	Templating engines	10.9k / 389	15	13.3	0.0	0/3
`json`	`orjson`	JSON Serialization	N/A / 6.9k	269	96.7	57.6	258/291
`logging`	`loguru`	Logging	N/A / 21.7k	114	85.1	72.8	64/93
`lxml`	`BeautifulSoup`	HTML/XML parsing	2.8k / N/A	32	59.4	0.0	0/21
`pandas`	`polars`	Data analysis	45.5k / 33.8k	0	-	-	-
`pathlib`	`pyfilesystem2`	Filesystem abstraction	N/A / 2.0k	21	52.4	28.6	1/15
`cryptography`	`pycryptodome`	Cryptography	7.0k / 3.0k	79	48.1	6.3	3/46
`requests`	`httpx`	HTTP clients (async, sync)	52.9k / 14.1k	65	76.9	12.3	39/63
`time`	`pendulum`	Date/time	N/A / 6.4k	60	78.3	11.7	35/73

model), relying on prior work showing that more generation attempts can compensate for smaller models [46]. For synthesis, we use `gpt-4.1` [36], a current state-of-the-art model with more advanced capabilities and larger context window (1M tokens), required for our large system prompt and chained interactions.

Migration tasks. We evaluate on 10 migration tasks across 20 `Python` libraries; the left-hand-side of Table 1 summarizes. These tasks were selected to span a diverse range of popular libraries and their alternatives across multiple domains, which prior work has also targeted [14, 21, 35, 41]. We focus on `Python` because there is a well-documented lack of refactoring tools for `Python` [30], despite the fact that it is among the most popular programming languages [6]; this also makes it a good target for LLM-based tooling [24].

Settings. We generate 100 use cases per migration task $(\mathcal{S}, \mathcal{T})$. We generated 5 source implementations $\tilde{s}_{j,k}$ for each use case $C_j \in C$; 5 test files $\tilde{t}_{j,k,\ell}$ per source implementation $\tilde{s}_{j,k}$; and 5 migration attempts $\tilde{m}_{j,k,r}$ per source implementation $\tilde{s}_{j,k}$. This initially yields 500 implementations, 2,500 test files, and 2,500 migration attempts per task; these are validated and filtered as described in Section 3.2. We run our synthesis pipeline on each validated triple to attempt to generate a `PolyglotPiranha` script $\mathcal{P} = (\mathcal{R}, \mathcal{E})$. We allow the agent up to 10 iterations to produce the script.

We chose these numbers (5 samples per generation step and up to 10 agent iterations) to balance cost and coverage: based on model pricing, this setup allowed us to stay within a $100 total budget while still producing meaningful results. We sample multiple migration attempts per implementation because migration is the hardest generation step; multiple attempts increase the chance of success [46].

5.2 RQ1: End-to-end effectiveness

Methodology. We first evaluate our synthesis pipeline end-to-end. We report, for each migration task, the number of valid triples generated over all use cases (Table 1, 5th column, "Valid triples"); this is the output of data generation (Section 3). Success rate (for SPELL, Table 1 column 6) reports the percentage of valid examples on which script synthesis produces a successful `PolyglotPiranha` program. We investigate synthetic data quality and diversity in Section 5.6. To evaluate script generalizability, we test each $\mathcal{P}_{j,k}$ (from triple $(s_{j,k}, t_{j,k,\ell}, m_{j,k,r})$) on *sibling implementations* $\{s_{j,i} : i \neq k\}$ of the same use case C_j. Sibling implementations are *different* implementations of the *same*

Table 2. Inferred migration scripts applied to real-world repositories. Each row represents the application of a migration script to a repository that uses the source library. **Stars** and **KLoC** refer to the GitHub popularity and codebase size. **Test Cov.** is the line-level test coverage of the repository's executing test suite. **Rewrites** is the number of times a `PolyglotPiranha` rule triggered in the project. Passing Tests (Before/After/%): Number of passing test cases pre- and post-migration, with the percentage of tests preserved (post/pre), indicating basic functionality is maintained.

						Passing Tests		
Migration Task	**Repository**	**Stars**	**KLoC**	**Test Cov.**	**Rewrites**	**Before**	**After**	**%**
`json → orjson`	`BIM2SIM/bim2sim`	57	33.8	37%	46	173	149	86%
	`kaapana/kaapana`	196	69.8	56%	317	45	43	96%
	`SoCo/SoCo`	1500	17.4	51%	7	233	233	100%
`logging → loguru`	`fastqe/fastqe`	172	0.7	38%	16	2	2	100%
	`mie-lab/trackintel`	229	11.2	74%	14	411	411	100%
	`pywbem/pywbem`	42	171.7	42%	16	2499	0	0%
`lxml → BeautifulSoup`	`acl-org/acl-anthology`	528	19.4	36%	111	433	433	100%
	`pyxnat/pyxnat`	73	7.7	55%	8	62	62	100%
	`w3af/w3af`	43	201.8	57%	10	2547	2547	100%
`cryptography → pycryptodome`	`aomail-ai/aomail-app`	134	2.0	67%	1	13	11	85%
	`IBM/qpylib`	32	2.0	99%	4	139	127	91%
	`DMcP89/harambot`	32	1.6	49%	8	8	0	0%
`requests → httpx`	`Kildrese/scholarBibTex`	33	0.1	98%	5	1	1	100%
	`wjohnson/pyapacheatlas`	175	13.9	60%	22	128	128	100%
	`databricks/databricks-sdk-py`	433	108.6	41%	23	157	157	100%
`time → pendulum`	`clld/clld`	57	9.1	97%	4	308	308	100%
	`fbpic/fbpic`	192	25.5	36%	43	6	5	83%
	`qutip/qutip`	1800	41.2	43%	29	1585	147	9%

use case, generated via separate LLM samples; they share the same functional goal but may differ in code structure, variable names, or API usage patterns. Testing on siblings thus measures whether a synthesized script captures the *general* transformation pattern rather than overfitting to one specific code instance. We report how many siblings are successfully migrated by the generated scripts, per the tests.

Results. Table 1 shows results. Spell successfully generated valid migration triples for nine of ten tasks, producing 870 filtered, valid triples in total, an average of 87 per task across all ten tasks (the `pandas → polars` exception is discussed in Section 5.6). Using these validated examples, Spell's migration synthesis script succeeded, on average, 61.6% of the time, using a single trial. This rate could likely improve further with test-time scaling [46]; we leave this to future work, given cost.

The last column of Table 1 shows that the scripts migrated 63.3% of sibling implementations (491/775). Performance was particularly strong for simple, one-to-one API replacements like `json → orjson` (88.6 %) while migrations involving embedded DSLs (`jinja2 → mako`, `lxml → beautifulsoup`) generalize less well.

These libraries support dynamic webpage generation and embed templating languages within `Python` string literals, which the `PolyglotPiranha` engine cannot transform; this is a limitation of the `PolyglotPiranha` language itself [25]. Despite this constraint, our synthesized rules effectively capture common usage patterns across most migration tasks.

5.3 RQ2: Real-world applicability

Methodology. To evaluate whether our inferred migration scripts generalize beyond synthetic examples, we applied them to 18 Python projects hosted on GitHub. We used `Sourcegraph` [1] to collect a convenience sample of codebases containing source API calls that were present in

our generated scripts, sorting them by star, and prioritizing codebases with test suites. Due to the high engineering effort required to make each project build and test successfully, we conducted this evaluation for only six of the ten migration tasks.

Because our LLM budget was deliberately limited, the synthesized scripts cover only a subset of each library's API, around 9% of the source API surface on average (as we explain in Section 5.6). Crucially, we synthesized these scripts *first and independently of any repository*, then searched for projects that exercise the source APIs they cover, retaining three per task. The repositories were therefore not chosen to be easy or to resemble the generated examples; a real project migrating under a script fixed in advance is precisely the evidence that the script generalizes beyond the synthetic triple it was distilled from.

SPELL produces more of a script portfolio rather than a single monolithic script per library pair as each successful synthesis from a validated migration triple yields a script specialized to that example's API usage pattern. The manual review is therefore a *selection among many candidate scripts*, not the tuning of one script to fit each codebase. For each project, we inspected its pre-migration source-library usages and picked a script whose match patterns covered at least one observed usage; scripts whose patterns were absent simply produce no rewrites, so this step merely identifies a syntactically applicable script. The evaluation thus measures transfer of an already-synthesized script to compatible real-world usages, not automated retrieval or complete coverage of a library's API. The fact that this selection is currently manual does not undercut SPELL's efficiency, as a script is validated once and then applied deterministically across the whole codebase. SPELL's cost therefore scales with the migration's complexity, not the codebase's size.

We containerized execution using `Dockerfiles` based on each project's documented setup, ran tests to establish baseline functionality, then applied the selected `PolyglotPiranha` script and re-ran tests. We updated `requirements.txt` to include target dependencies, but retained source dependencies, since these codebases exercise many source APIs our scripts do not target. We did not measure transformation time, as `PolyglotPiranha`'s engine executes near-instantaneously. These non-trivial codebases typically span hundreds of files and thousands of lines of code.

Results. Table 2 reports the number of rewrites (i.e., the number of times a particular `Polyglot-Piranha` rule was triggered to transform code) per project, as well as the number of passing tests before and after migration. By checking whether pre-existing tests still pass, we validate that the functionality maintainers deemed important is preserved. The test suites of the subject repositories achieve between 36% and 99% line coverage (median 53%, average 58%), providing a non-trivial signal of functional correctness, though coverage varies considerably across projects.

In many cases, all tests passed post-migration, despite scripts being inferred from synthetic examples. For example, for the `logging` → `loguru` migration, the inferred script preserved test behavior in both the `trackintel` and `fastqe` projects and triggered a substantial number of rewrites. Similarly, SPELL's scripts for `json` → `orjson` and `requests` → `httpx` applied dozens to hundreds of edits across real-world projects while preserving functional behavior. Upon manual inspection, we found that test failures were typically due to incomplete migrations: the scripts handled part of the migration correctly but required further refinement or additional transformations.

Note that overall coverage does not indicate whether the tests reach the specific code being migrated. For a subset of projects, we inspected in detail whether the test suite executes the code rewritten by our scripts. In `clld/clld`, the suite executes 3 out of 4 rewritten executable statements; in `databricks/databricks-sdk-py`, the test suite exercises 15 out of 23 migrated sites. Thus, for these projects, the existing tests provide direct evidence about the behavior they check. In contrast, only 1 of the 16 rewritten sites in `fastqe/fastqe` is exercised by the test suite. This reflects the fact that the package's test coverage was already low (38%). For `acl-org/acl-anthology`, all rewrites

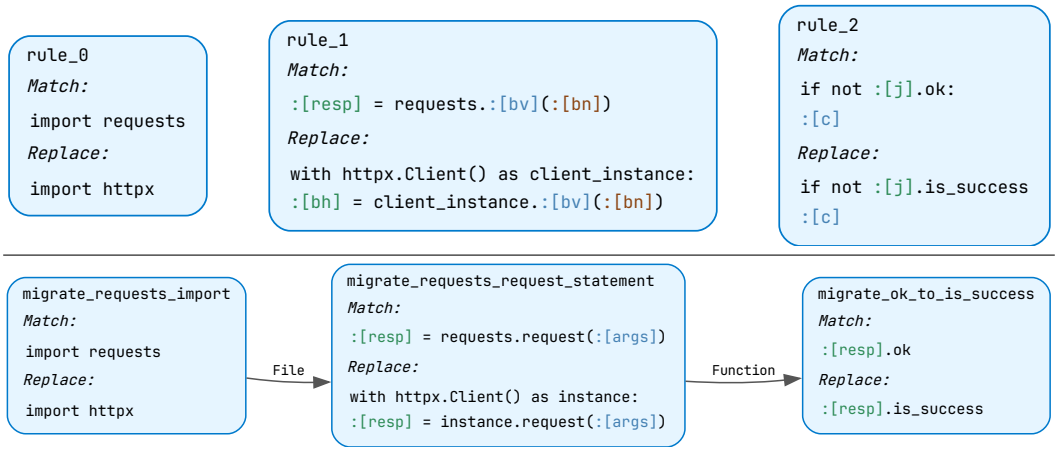


Fig. 5. A program generated by Melt (top) and Spell (bottom) from a migration example for `requests` → `httpx`. Melt's rules contain generic placeholder names and overabstracts; Spell's script includes scoped application and semantically meaningful names.

are in stand-alone command-line scripts under `bin/`, which the tests do not cover. Here, passing tests are not indicative of a successful migration. Additional tests would be necessary to validate these changes, which is out of scope for our approach.

5.4 RQ3a: Comparison with MELT

Methodology. We compare our synthesis approach with Melt [41], a state-of-the-art tool for automated `Python` refactoring. Melt is one of the few tools designed for Python and is built on the transformation DSL `comby` [49, 50] (see Section 7). Given a pair of code snippets representing a before/after change, Melt uses anti-unification to infer a single `comby` rewrite rule that generalizes the change. Unlike Spell, Melt produces individual, independent rules without composition, scoping, or cascading—each rule is a standalone match-replace pattern applied globally. In its original setting, Melt operates on migration pairs mined from developer pull requests. Since our migration tasks are not associated with existing pull requests (cross-library migrations rarely are), we run Melt on Spell's generated migration triples, giving both tools the same input. This isolates the comparison to the synthesis step: both tools receive identical examples, but differ in how they produce transformation scripts.

Results. Table 1, columns 6 and 7, compares the two approaches. Spell outperforms Melt on all tasks for which valid triples are available, often by a large margin. It achieves a 61.6% (average) success rate compared to Melt's 22.9%, despite both tools starting from the same validated examples. The fact that Melt can extract transformation rules from many of the synthetic examples provides additional evidence of their quality, and suggests the data generation pipeline may enhance other migration inference tools. However, Melt's rules are created in isolation. Spell's agentic pipeline, by contrast, orchestrates these kinds of rules into coherent transformation strategies, effectively taking advantage of the expressive power of the `PolyglotPiranha` language. Figure 5 illustrates this difference: Spell synthesizes interconnected rules with explicit scoping (per-function, per-file) that enable cascading transformations without overgeneralization; Melt produces disconnected global rules with generic placeholders, which can prove especially problematic on large codebases [25].

Table 3. RQ3b: Spell vs. Direct LLM Migration (GPT-4.1). Test pass rate and number of rewrites per repository (for Spell, `PolyglotPiranha` rule applications; for the baseline, changed call sites). Spell wins 10 of 18 repositories, ties 7, and loses 1. The difference is statistically significant (Wilcoxon signed-rank, $p = 0.0004$, Cohen's $d = 0.96$). **S** = Spell wins, **B** = Baseline wins, = = tie (≤5pp difference).

Migration Task	Repository	Spell		LLM Baseline		
		Pass	Rewrites	Pass	Rewrites	
`requests → httpx`	`scholarBibTex`	100%	5	100%	2	=
	`pyapacheatlas`	100%	22	0%	6	**Spell**
	`databricks-sdk-py`	100%	23	0%	19	**Spell**
`logging → loguru`	`fastqe`	100%	16	0%	1	**Spell**
	`trackintel`	100%	14	0%	4	**Spell**
	`pywbem`	0%	16	0%	10	=
`json → orjson`	`bim2sim`	86%	46	31%	17	**Spell**
	`kaapana`	96%	317	0%	133	**Spell**
	`SoCo`	100%	7	95%	4	=
`time → pendulum`	`fbpic`	83%	43	82%	4	=
	`qutip`	9%	29	0%	18	**Spell**
	`clld`	100%	4	31%	4	**Spell**
`lxml → beautifulsoup`	`pyxnat`	100%	8	0%	6	**Spell**
	`w3af`	100%	10	98%	9	=
	`acl-anthology`	100%	111	0%	86	**Spell**
`cryptography → pycryptodome`	`qpylib`	91%	4	99%	2	**Baseline**
	`aomail-app`	85%	1	85%	1	=
	`harambot`	0%	8	0%	3	=
Average/Total		**81%**	**684**	**34%**	**329**	

These results validate that (1) the generated synthetic examples provide a useful independent signal for migration inference, (2) the agentic approach to rule composition meaningfully improves over the current state-of-the-art, with intelligent orchestration of inferred rules importantly contributing to migration synthesis.

5.5 RQ3b: Comparison with Direct LLM Migration

Methodology. To evaluate whether distilling migration knowledge into reusable scripts provides advantages over directly using an LLM to perform migrations, we compare Spell against a direct LLM-based migration baseline. For each of the 18 real-world repositories evaluated in Section 5.3, we prompt the LLM to migrate each source file, that imports the library, from the source library to the target library. We apply the initial filtering by imports to reduce the cost of the migrations, and use temperature 0 to reduce non-determinism. Additionally, we use the same model (`gpt-4.1`) for the baseline to ensure a fair comparison. We measure test pass rates before and after migration and compare with Spell rates reported before.

Results. Table 3 summarizes the results. Spell achieves a mean test pass rate of 81% after migration, compared to 34% for the direct LLM baseline. Spell outperforms the baseline on 10 of 18 repositories, ties on 7, and loses on only 1 (`qpylib`, 99% vs. 91%). The baseline fails most severely on large migrations: in `kaapana` and `acl-anthology`, where it rewrote 133 and 86 call sites, and in `databricks-sdk-py` (108.6 KLoC), the pass rate drops to 0%, as file-by-file migration provides no mechanism for cross-file consistency. Spell's `PolyglotPiranha` scripts also perform more rewrites overall (684 vs. 329), as each rule applies uniformly to every matching occurrence, whereas the LLM

Table 4. Quality of generated migration examples: use case success rates, API diversity, and test coverage

Migration Task	use cases (out of 100)		Distinct APIs		Avg. Coverage		Validated
	Implemented	Migrated	Source	Target	Source	Target	Examples
`argparse → click`	74	72	11	16	0.89	0.93	215
`jinja2 → mako`	24	14	9	4	0.86	0.86	15
`json → orjson`	96	96	4	3	0.70	0.70	269
`logging → loguru`	58	52	12	6	0.79	0.79	114
`lxml → beautifulsoup`	59	23	11	9	0.75	0.75	32
`pandas → polars`	58	2	5	4	0.43	0.43	0
`pathlib → pyfilesystem2`	64	23	10	17	0.71	0.70	21
`cryptography → pycryptodome`	72	52	59	25	0.73	0.72	79
`requests → httpx`	64	60	8	10	0.66	0.64	65
`time → pendulum`	81	55	9	34	0.61	0.61	60
Average	**65.0**	**44.9**	**13.8**	**12.8**	**0.71**	**0.71**	**87.0**

may leave some usages untouched. The two approaches tie on small migrations (`scholarBibTex`, `w3af`, `SoCo`) and both fail on the hardest cases (`pywbem`, `harambot`), where the required semantic changes exceed what either method can handle.

Beyond correctness, a key qualitative difference is that Spell produces *reusable, inspectable artifacts*: a developer can review the `PolyglotPiranha` script to understand exactly what will change before applying it, version-control the script alongside the codebase, and reapply it if the migration is rolled back or extended. Direct LLM migration provides none of these properties as each file is transformed independently with no guarantee of uniformity or auditability, and the transformation logic is not captured in any persistent artifact.

5.6 RQ4: Data Quality

Methodology We characterize the quality of the synthetically generated data across several dimensions:

(1) **Use case implementation:** the number of the 100 generated use cases $C_j \in C$ that yields at least one implementation $s_{j,k}$ that passes at least one generated test harness.
(2) **Use case migration:** the number of implemented use cases that are migrated to the target library (how many use cases have at least one s_i that is associated with a success m_i).
(3) **API diversity:** number of distinct API methods used from both $\mathcal{S}$ and $\mathcal{T}$.[3] We compute this using the Jedi static analyzer [18] on validated triples $(s, t, m) \in \mathcal{M}$.
(4) **Test coverage:** via line-level coverage of the generated tests on both source and target implementations (coverage must be at least 0.6, per our definition of validity).
(5) **Validated examples:** The total number of validated examples from all generated examples, post-filtering; this determines the training data available for script synthesis.

Together, these metrics characterize the quality of the generated synthetic data, in terms of how well they cover or represent a migration task, and the degree to which they are adequately tested.

Results. Table 4 shows results across all data quality metrics. Spell successfully implemented the majority of generated use cases, with an average of 65 out of 100 use cases yielding at least one working implementation. Success rates ranged from 24 (`jinja2 → mako`) to 96 (`json → orjson`). Upon manual inspection of `jinja2 → mako`, we found that most test code for this task used string comparisons for expected output, which the model struggled to get right. These tests were also not

[3]Due to `Python`'s dynamic typing, this heuristic should be considered a lower bound.

transferable between libraries, as each uses a different string representation to denote placeholders in the templates.

From the pool of use cases implemented at least once, SPELL successfully migrated an average of 44.9 use cases while maintaining functional equivalence per the generated tests. The outlier is `pandas → polars` with only 2 successful migrations despite 58 successful implementations. Manual inspection revealed that generated test harnesses relied on pandas-specific constructs (e.g., `DataFrames`) that were incompatible with `polars`.

Our analysis of API diversity using Jedi identified an average of 13.8 distinct source library APIs and 12.8 target library APIs per migration task. These likely correspond to the most frequently used APIs that the underlying LLM encountered during pre-training [24]. The `cryptography → pycryptodome` task showed the highest diversity (59 source, 25 target APIs), while simple tasks like json ⟶ orjson used fewer APIs (4 source, 3 target).

To contextualize these counts, we measure each library's public API surface by introspecting submodules via `pkgutil`[4] and counting all non-private callables, including public methods on top-level classes. The generated examples exercise between <1% and 32% of each library's source API surface (9% on average), with compact standard-library modules like `time` (9/28) and `json` (4/25) showing the highest coverage and large frameworks like `pandas` (5/5868) the lowest. This distribution aligns with prior empirical evidence that API usage is long-tailed and that many libraries expose substantial unused surface [39, 56]. In practice, developers could increase coverage by supplying full API documentation directly in the prompt, enabling the model to target APIs beyond those encountered during pre-training.

The generated tests achieved an average coverage of 71% for both source and target implementations, well above our 60% filtering threshold. Coverage ranged from 61% (`time → pendulum`) to 89%/93% (`argparse → click`). This suggests that tests meaningfully exercise implementation logic, while leaving room to improve; LLMs are known to struggle with unit test generation [42].

After applying all filtering criteria, SPELL produced an average of 87 validated migration triples per task across all ten tasks (870 in total; `pandas → polars` produced none). Overall, these examples are testable and diverse (covering an average of 13–14 distinct APIs per library), but also exhibit strong coverage (71% on average) on both source and migrated code. This supports their use as input for downstream synthesis.

6 Discussion and Limitations

Why Distill into Scripts Rather Than Use LLMs Directly? A natural question is whether the elaborate distillation pipeline is warranted when LLMs can attempt migrations directly. We argue that the two approaches serve complementary but distinct goals. Direct LLM migration is convenient for one-off, small-scale changes, but it provides no guarantee of uniformity, offers no inspectable artifact for review, and requires re-invoking the model—at non-trivial cost and latency—for every file. In contrast, SPELL's distilled scripts: (1) apply the *same* transformation deterministically across an entire codebase, ensuring consistency; (2) produce inspectable, version-controllable artifacts that developers can audit, modify, and reuse; (3) execute near-instantaneously via the `PolyglotPiranha` engine, with no runtime LLM dependency; and (4) can serve as high-quality training data for improving future migration models.

This approach mirrors established industry practice for large-scale migrations. For example, Uber migrated more than 2 million daily Spark jobs from Spark 2.4 to Spark 3.3 by using `Polyglot-Piranha` to apply predefined rewrite rules across over 2,100 applications, achieving 85% automatic migration with minimal manual intervention [51]. Similarly, Uber migrated more than 1 million

[4] https://docs.python.org/3/library/pkgutil.html

lines of JUnit 4 tests to JUnit 5 in two weeks using OpenRewrite [34]. In both cases, engineers first synthesized reusable transformation scripts and then applied them uniformly across the codebase. Spell automates the most difficult step of this workflow: deriving such scripts automatically.

Beyond performance and inspectability, scripts provide a deeper advantage in terms of validation. When an LLM migrates files independently, every generated output is a distinct artifact that **must be reviewed and validated on its own merits**. As a result, the validation burden grows with the size of the codebase. In contrast, a transformation script is validated once: because it applies a deterministic rewrite to every matching instance, correctness becomes a property of the script itself rather than of each individual application. Spell therefore shifts migration validation from a cost that scales with codebase size to one that scales with the intrinsic complexity of the migration.

Quality and Representativeness of Synthetic Data. Our pipeline relies on synthetic examples generated by a comparatively small model (`gpt4o-mini`) to keep the cost of data generation manageable. We show that this data is sufficient for our approach and is also usable by a previous migration inference approach. However, programs generated by LLMs are typically biased towards especially well-used parts of each API. Corner-case behaviours, error-handling paths, and rarely used configuration parameters are likely under-represented. As larger models trained on more data become cheaper, we expect both the breadth of API usage and correctness and quality of the generated programs to improve. A complementary direction is to augment the use case –implementation prompts with retrieval over real code repositories. This could be especially helpful for a use case enabling a client to articulate relevant API methods to guide sample generation, which could improve coverage of migration rules on real-world repositories.

Illustrative Analysis of Successful and Unsuccessful Syntheses. To better understand the strengths and limitations of Spell, two authors jointly inspected successful and unsuccessful examples from a subset of the evaluated migrations, focusing on those with the highest and lowest synthesis success rates. Below, we present examples that illustrate these observations. This informal analysis is qualitative and does not make claims about the frequency or representativeness of the observed patterns.

In this discussion, an unsuccessful outcome is one in which Spell synthesizes rules that `PolyglotPiranha` applies, but the transformed code does not pass the shared test suite.

Illustrative Successful Syntheses: Structural API Replacements. The successful examples illustrate migrations that can be reduced to syntactic rewrite rules over the module's public surface, including import statements, constructor calls, and method invocations whose names and argument order change predictably between the two libraries. The `json`→`orjson` migration illustrates this pattern at 96.7% synthesis success across 269 task instances. For example, one of the scripts synthesized for this migration consists of four rules chained under a single file-scoped edge: (1) a seed rule that rewrites `import json` to `import orjson`; (2) a context-manager rule that transforms `with` statements into the binary-mode equivalent; (3) an exception-name rule mapping; and (4) a symmetric write rule converting `json.dump(cfg, fh, indent=4)` into `fh.write(orjson.dumps(cfg))`. Every change is purely syntactic: no understanding of data flow is required, and the pattern metavariables match concrete identifiers at rewrite time without needing to track type information.

A somewhat more complex example comes from the `logging`→`loguru` migration (85.1% success). Here the synthesized rules must handle both a namespace change and a constructor-to-sink transformation: `import logging` becomes `from loguru import logger`, and the multi-argument calls must be rewritten, even though the argument mappings are not one-to-one. Another example comes from the `requests`→`httpx` migration (76.9% success), where one synthesized script captures a structural idiom change by wrapping a regular API call in a context-managed

construct: `requests.request(...)` → `with httpx.Client() as c: c.request(...)`. The rule uses a Function-scope chained edge so dependent predicate rewrites apply only within already-transformed functions.

Illustrative Unsuccessful Syntheses: Embedded DSLs and Semantic Incompatibility. One observed failure pattern occurs when the transformation target is not a syntactic fragment of the Python AST but instead a string literal whose interior encodes a domain-specific language. The `jinja2→mako` migration illustrates this pattern: Jinja2 templates embed directives such as `{{ variable }}`, `{% if cond %}` inside Python string literals, while Mako uses syntactically incompatible equivalents. `PolyglotPiranha` operates on Python source code; it cannot parse or rewrite string literal contents, so the synthesized rules can only capture the surrounding Python API calls while leaving the template body untouched. As a result, the migrated code passes API-level assertions but fails any test that renders a template. The overall synthesis success rate for `jinja2→mako` is 13.3% (2 of 15 validated triples; Table 1).

The `pathlib→pyfilesystem2` migration (52.4% success) illustrates a different failure pattern: abstraction-level mismatch. `Pathlib` models a filesystem path as a value (`pathlib.Path`) exposing attribute-style navigation (`p / "subdir"`, `p.glob("*")`), whereas `pyfilesystem2` separates a filesystem handle (`fs.open_fs(...)`) from string path arguments passed to handle methods (`handle.listdir('/')`). A single `Pathlib` call site can therefore require coordinated rewrites across callee and callers: a helper accepting a `Path` argument must gain a new `fs` handle parameter, and all call sites in other files must be updated accordingly. `PolyglotPiranha` metavariables match lexical patterns within a single syntactic context; they cannot introduce new formal parameters into a function signature and simultaneously propagate a new binding to call sites in other files. For example, one unsuccessful script synthesized for this migration produces 14 rules with inconsistent variable naming across scopes, leading to test failures even when `PolyglotPiranha` applies the rules without error.

Applicability. Section 5 shows that the technique is promising for some migration tasks (e.g., when the code manipulates standard data types or produces well-defined outputs that are easy to compare even across libraries) and inadequate in others. A main limitation in real-world usage is the many variations of code found in open-source projects. The atomic rules inferred by MELT capture concrete diff hunks faithfully but can be either too specific, failing to match slight syntactic variations, or too general, introducing false positives. SPELL's agentic loop alleviates some of these issues by refining and testing rules, yet it remains fundamentally example-bound: if a usage pattern does not appear in any migration triple, the resulting script will not handle it. `Pycraft` [13]'s technique on rule generalization using LLMs could be integrated into our workflow to generate further equivalent script variations for a broader application. Future work can also complete partial migrations by incorporating project-specific API usage patterns to refine migration scripts and by using failing test feedback to fix remaining issues.

More broadly, SPELL supports a spectrum of use. At one extreme, a user with an unlimited budget could synthesize migrations that cover all of a library's APIs. At the other extreme, a user could target a single project, generating use cases for exactly the APIs that project relies on. Our evaluation deliberately sits in the middle. We synthesized scripts under a limited budget, so they cover only a subset of each library's APIs, and we then applied them to repositories where they were applicable. This choice stresses generalization, because we fixed the scripts before we selected the repositories. In practice, a deployment can move toward either extreme: we can broaden coverage by increasing the generation budget, or we can sharpen it by seeding use-case generation with a target project's own API usage.

Generated tests. A core pillar of Spell's validation process is the use of automatically generated tests to assess the correctness of both implementations and their migrated counterparts. While this enables large-scale validation without manual effort, it introduces some limitations. First, the quality of the tests is inherently tied to the generative model's ability to produce robust and meaningful test harnesses. Although we mitigate this by sampling multiple test files per implementation and applying a filter based on coverage as explained in Section 3.1.3, the tests may still be shallow or fail to exercise edge-case behavior. In particular, tests confirm only basic functional equivalence (e.g., matching outputs), so they may miss subtle behavioral divergences or side effects. Second, our approach assumes that passing the same test suite is a sufficient proxy for semantic equivalence. While this assumption is reasonable for many cases, it is not formally guaranteed. The test suites may lack completeness or specificity. Third, our coverage-based filtering helps ensure that the tests exercise a non-trivial portion of the implementation logic, but it does not guarantee semantic soundness. For instance, tests that exercise a large portion of the code but contain weak assertions (e.g., asserting only that no exceptions are raised) may still pass despite incorrect behavior. Other validation techniques (e.g., mutation testing [23]) could help mitigate this issue.

Real-world test suites. The real-world study (Section 5.3) instead relies on each project's own human-written tests, so its evidence does not depend on our test generation. It is important, however, to also consider whether these tests actually cover the migrated code: a passing suite validates a migration only to the extent that it exercises the rewritten call sites, and some projects may have no tests over the code paths that were changed. To apply Spell in practice, a developer should therefore check that the migrated code is exercised by tests, adding tests where it is not, before relying on a passing suite to conclude that the project still works after migration.

7 Related Work

Code Transformation Toolsets. Code transformation toolsets vary widely. Imperative frameworks for AST rewriting include Java's `errorprone`[2] and `OpenRewrite`[9]. `clang libtooling`[8] provides declarative matching via AST matchers, along with a framework for code rewriting.

Purely declarative tools for code transformation also exist, for example, `coccinelle` [32], `GritQL` [47], `ast-grep` [4], `comby` [50], and `PolyglotPiranha` [25]. `comby` is the intellectual predecessor of `PolyglotPiranha`, which we use in this paper. `PolyglotPiranha` builds on the `comby` model by allowing rules to be composed for fine-grained and scoped rewrites using cascading transformations (edges), explicit scopes, and filters. We chose to represent our migration scripts in `PolyglotPiranha` because of its demonstrated success in industrial-scale applications [25] and its syntactic simplicity, which makes it amenable for synthesis.

Transformation by Example. Many tools infer migration rules from examples [3, 14, 16, 17, 26, 31, 33, 54]. Most of these techniques assume access to projects that were manually migrated from one library to another, but such examples are rare [48]. Finding them typically requires mining large repositories and reconstructing change histories [20]. Spell can complement these approaches by automatically distilling code migration knowledge from LLMs into examples that can serve as training data for such tools, reducing the need to rely on manually curated migration histories.

Only recently have these tools begun to leverage production-ready transformation engines like Melt [41], `PyEvolve` [14], and `TCInfer` [26, 45], which express migration logic in the `comby` language. Among them, Melt is the most closely related to our work: it targets breaking changes, while Spell focuses on cross-library migrations. We use Melt 's inference algorithm to generate an initial rule set, which we then refine using our agentic approach to produce complete migration scripts in `PolyglotPiranha`. Our method could also benefit from recent work on transformation rule generalization, particularly `Pycraft` [13], which generates variants of individual rules for

broader applicability, addressing a known limitation of both `PolyglotPiranha` and `comby` (i.e., how to generalize rules beyond the specific example they were inferred from).

LLMs for Code Transformation. Recent developments show growing interest in using LLMs for automating code migration. Industry tools such as `Amazon Q Developer` [44] and reports of LLM-assisted Java upgrades [57] reflect this trend.

`DeepMig` [11] trains a sequence-to-sequence model on migration pairs mined from open-source repositories to perform both library-level and API-level migrations. While effective when sufficient training data is available, `DeepMig`'s reliance on supervised learning over mined corpora limits its applicability to library pairs for which such data exists—precisely the data scarcity problem SPELL is designed to address. SPELL requires no preexisting migration data and produces reusable, inspectable transformation scripts rather than direct code translations.

Islam et al. [21] demonstrated that LLMs can achieve high success rates on individual file-level migrations in real-world projects, providing encouraging evidence that LLMs encode substantial migration knowledge. However, direct LLM-based migration offers no mechanism for ensuring uniformity across a codebase, produces no reusable artifacts, and requires re-invoking the model for each file. SPELL complements this work by showing that such knowledge can be *distilled* into deterministic, reusable scripts suited for large-scale, auditable migrations.

`SOAR` [35] is one of the earliest approaches to leverage language models for migration. It uses API documentation to infer correspondences between functions in the source and target libraries, then used to migrate client code via program synthesis. Unlike SPELL, it does not generate migration examples or transformation scripts. Both approaches use language models to capture migration knowledge: SPELL through prompting and pretraining, `SOAR` through embedding similarity. MELT [41] uses a pull request that introduces a breaking change to prompt an LLM to generate transition examples from the old to the new version, combining model knowledge with example code and tests. SPELL uses a more elaborate self-instruction loop to generate use cases, implementations, and migrations, extending MELT 's approach to a broader domain.

8 Conclusion

Library migration is a frequent, labor-intensive task for developers. We present SPELL, a system that extracts migration knowledge from large language models and converts it into reusable `PolyglotPiranha` scripts. Instead of relying on manually migrated corpora, SPELL prompts LLMs to generate code and tests for both source and target libraries, then distills generalized transformations using anti-unification and agentic synthesis. The resulting scripts are compatible with modern refactoring workflows and can be inspected, versioned, and applied across large codebases.

We evaluated SPELL on ten Python migration tasks, generating 870 validated migration examples. With one synthesis attempt per example, SPELL produced correct scripts for 61.6% of cases and migrated 63.3% of sibling implementations, consistently outperforming MELT. Applied to 18 real-world open-source repositories, SPELL made hundreds of edits while preserving test behavior in most cases. These results show that LLMs encode actionable migration knowledge that can be extracted, validated, and formalized into maintainable transformation logic.

References

[1] [n. d.]. Sourcegraph. https://sourcegraph.com/. Accessed on July 18, 2025.
[2] Edward Aftandilian, Raluca Sauciuc, Siddharth Priya, and Sundaresan Krishnan. 2012. Building Useful Program Analysis Tools Using an Extensible Java Compiler. IEEE Computer Society, Riva del Garda, Italy, 14–23. doi:10.1109/SCAM.2012.28
[3] Jesper Andersen and Julia L. Lawall. 2008. Generic Patch Inference. In *Proc. International Conference on Automated Software Engineering (ASE)*. IEEE Computer Society, 337–346. doi:10.1109/ASE.2008.44
[4] Astgrep. 2024. *ast-grep: write code to match code.* Open-source. https://ast-grep.github.io Accessed: 2024-03-06.

[5] Christopher Bogart, Christian Kästner, James D. Herbsleb, and Ferdian Thung. 2016. How to break an API: cost negotiation and community values in three software ecosystems. In *Proc. ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE)*, Thomas Zimmermann, Jane Cleland-Huang, and Zhendong Su (Eds.). ACM, 109–120.

[6] TIOBE Software BV. 2025. TIOBE Index. https://www.tiobe.com/tiobe-index/. Accessed on May 5, 2025.

[7] Ned Chapin, Joanne E Hale, Khaled Md Khan, Juan F Ramil, and Wui-Gee Tan. 2001. Types of software evolution and software maintenance. *Journal of Software Maintenance and Evolution: Research and Practice* 13, 1 (2001), 3–30.

[8] Clang. 2024. *Clang LibTooling*. Accessed: 2024-03-06.

[9] OpenRewrite Contributors. 2025. OpenRewrite: Refactoring Made Safe. https://docs.openrewrite.org/. Accessed: 2025-03-27.

[10] Cleidson RB De Souza, David Redmiles, Li-Te Cheng, David Millen, and John Patterson. 2004. How a good software practice thwarts collaboration: the multiple roles of APIs in software development. *ACM SIGSOFT Software Engineering Notes* 29, 6 (2004), 221–230.

[11] Juri Di Rocco, Phuong T Nguyen, Claudio Di Sipio, Riccardo Rubei, Davide Di Ruscio, and Massimiliano Di Penta. 2025. DeepMig: A transformer-based approach to support coupled library and code migrations. *Information and Software Technology* 177 (2025), 107588.

[12] Danny Dig and Ralph E. Johnson. 2006. How do APIs evolve? A story of refactoring. *Journal of Software Maintenance: Research and Practice* 18, 2 (2006), 83–107.

[13] Malinda Dilhara, Abhiram Bellur, Timofey Bryksin, and Danny Dig. 2024. Unprecedented code change automation: The fusion of LLMs and transformation by example. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 631–653.

[14] Malinda Dilhara, Danny Dig, and Ameya Ketkar. 2023. PyEvolve: Automating Frequent Code Changes in Python ML Systems. In *Proc. ACM IEEE International Conference on Software Engineering (ICSE)*.

[15] Mattia Fazzini, Qi Xin, and Alessandro Orso. 2019. Automated API-usage update for Android apps. In *Proc. of the International Symposium on Software Testing and Analysis*, Dongmei Zhang and Anders Møller (Eds.). ACM, 204–215.

[16] Mattia Fazzini, Qi Xin, and Alessandro Orso. 2020. APIMigrator: an API-usage migration tool for Android apps. In *Proc. IEEE/ACM 7th International Conference on Mobile Software Engineering and Systems*, David Lo, Leonardo Mariani, and Ali Mesbah (Eds.). 77–80.

[17] Xiang Gao, Arjun Radhakrishna, Gustavo Soares, Ridwan Shariffdeen, Sumit Gulwani, and Abhik Roychoudhury. 2021. APIFix: Output-oriented Program Synthesis for Combating Breaking Changes in Libraries. In *Proc. ACM SIGPLAN Object-Oriented Programming, Systems, Languages & Applications (OOPSLA)*, Vol. 5. 1–27.

[18] David Halter. [n. d.]. Jedi: An autocompletion tool for Python. https://jedi.readthedocs.io/en/latest/. Accessed: April 25, 2025.

[19] Uber Technologies Inc. 2025. Piranha: A tool for refactoring code related to feature flag APIs. https://github.com/uber/piranha Accessed: 2025-04-10.

[20] Mohayeminul Islam, Ajay Kumar Jha, Ildar Akhmetov, and Sarah Nadi. 2024. Characterizing Python Library Migrations. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 92–114.

[21] Md Mohayeminul Islam, Ajay Kumar Jha, May Mahmoud, Ildar Akhmetov, and Sarah Nadi. 2025. Using LLMs for Library Migration. *arXiv preprint arXiv:2504.13272* (2025).

[22] Kevin Jesse, Toufique Ahmed, Premkumar T Devanbu, and Emily Morgan. 2023. Large language models and simple, stupid bugs. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 563–575.

[23] Yue Jia and Mark Harman. 2010. An analysis and survey of the development of mutation testing. *IEEE transactions on software engineering* 37, 5 (2010), 649–678.

[24] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. 2024. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515* (2024).

[25] Ameya Ketkar, Daniel Ramos, Lazaro Clapp, Raj Barik, and Murali Krishna Ramanathan. 2024. A Lightweight Polyglot Code Transformation Language. *Proc. ACM Program. Lang.* 8, PLDI (2024), 1288–1312. doi:10.1145/3656429

[26] Ameya Ketkar, Oleg Smirnov, Nikolaos Tsantalis, Danny Dig, and Timofey Bryksin. 2022. Inferring and Applying Type Changes. In *Proc. ACM IEEE International Conference on Software Engineering (ICSE)*. ACM, 1206–1218.

[27] Miryung Kim, Thomas Zimmermann, Robert DeLine, and Andrew Begel. 2018. Data scientists in software teams: state of the art and challenges. In *Proc. ACM IEEE International Conference on Software Engineering (ICSE)*, Michel Chaudron, Ivica Crnkovic, Marsha Chechik, and Mark Harman (Eds.). 585.

[28] Miryung Kim, Thomas Zimmermann, and Nachiappan Nagappan. 2012. A field study of refactoring challenges and benefits. In *Proc. ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE)*. ACM, 50.

[29] Raula Gaikovina Kula, Daniel M. Germán, Ali Ouni, Takashi Ishio, and Katsuro Inoue. 2018. Do developers update their library dependencies? - An empirical study on the impact of security advisories on library migration. *Springer Empirical Software Engineering (ESE)* 23, 1 (2018), 384–417.

[30] Maxime Lamothe, Yann-Gaël Guéhéneuc, and Weiyi Shang. 2021. A systematic review of API evolution literature. *ACM Computing Surveys (CSUR)* 54, 8 (2021), 1–36.

[31] Maxime Lamothe, Weiyi Shang, and Tse-Hsun Peter Chen. 2022. A3: Assisting Android API Migrations Using Code Examples. *IEEE Transactions on Software Engineering (TSE)* (2022), 417–431.

[32] Julia Lawall and Gilles Muller. 2018. Coccinelle: 10 Years of Automated Evolution in the Linux Kernel. In *USENIX Annual Technical Conference*, Haryadi S. Gunawi and Benjamin C. Reed (Eds.). USENIX Association, Boston, MA, USA, 601–614. https://www.usenix.org/conference/atc18/presentation/lawall

[33] Na Meng, Miryung Kim, and Kathryn S. McKinley. 2013. LASE: locating and applying systematic edits by learning from examples. In *Proc. ACM IEEE International Conference on Software Engineering (ICSE)*, David Notkin, Betty H. C. Cheng, and Klaus Pohl (Eds.). 502–511.

[34] Moderne. 2024. How Uber Migrated 1M Lines of JUnit in 2 Weeks On Demand. https://www.moderne.ai/content-library/how-uber-migrated-1m-lines-junit-2-weeks-on-demand. Moderne Engineering Blog.

[35] Ansong Ni, Daniel Ramos, Aidan Z. H. Yang, Inês Lynce, Vasco M. Manquinho, Ruben Martins, and Claire Le Goues. 2021. SOAR: A synthesis approach for data science API refactoring. In *Proc. ACM IEEE International Conference on Software Engineering (ICSE)*. IEEE, 112–124.

[36] OpenAI. 2025. *GPT-4.1: Introducing GPT-4.1 in the API.* https://openai.com/index/gpt-4-1/

[37] OpenAI. [n. d.]. OpenAI Python API Library. https://pypi.org/project/openai/. Accessed: April 22, 2025.

[38] Jeff H. Perkins. 2005. Automatically generating refactorings to support API evolution. In *Proc. ACM Workshop on Program Analysis for Software Tools and Engineering*. 111–114.

[39] Dong Qiu, Bixin Li, and Hareton Leung. 2016. Understanding the API usage in Java. *Inf. Softw. Technol.* 73, C (May 2016), 81–100. doi:10.1016/j.infsof.2016.01.011

[40] Daniel Ramos, Catarina Gamboa, Inês Lynce, Vasco Manquinho, Ruben Martins, and Claire Le Goues. 2025. Spell: Synthesis of Programmatic Edits using LLMs. https://figshare.com/s/08961f436816a434b77b. Artifact on Figshare.

[41] Daniel Ramos, Hailie Mitchell, Inês Lynce, Vasco Manquinho, Ruben Martins, and Claire Le Goues. 2023. MELT: Mining Effective Lightweight Transformations from Pull Requests. In *Proc. International Conference on Automated Software Engineering (ASE)*. IEEE, 1516–1528. doi:10.1109/ASE56229.2023.00117

[42] Nikitha Rao, Kush Jain, Uri Alon, Claire Le Goues, and Vincent J. Hellendoorn. 2023. CAT-LM Training Language Models on Aligned Code And Tests. In *38th IEEE/ACM International Conference on Automated Software Engineering, ASE 2023, Luxembourg, September 11-15, 2023*. IEEE, 409–420. doi:10.1109/ASE56229.2023.00193

[43] Baptiste Rozière, Marie-Anne Lachaux, Lowik Chanussot, and Guillaume Lample. 2020. Unsupervised Translation of Programming Languages. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (Eds.). https://proceedings.neurips.cc/paper/2020/hash/ed23fbf18c2cd35f8c7f8de44f85c08d-Abstract.html

[44] Amazon Web Services. 2025. Amazon Q Developer: Transform. https://aws.amazon.com/q/developer/transform/. Accessed: 2025-03-27.

[45] Oleg Smirnov, Ameya Ketkar, Timofey Bryksin, Nikolaos Tsantalis, and Danny Dig. 2022. IntelliTC: Automating Type Changes in IntelliJ IDEA. In *Proc. ACM IEEE International Conference on Software Engineering (ICSE)*. 115–119.

[46] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. 2024. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314* (2024).

[47] GritQL Team. 2025. GritQL: Code Transformation Query Language. https://grit.io/gritql/. Accessed: 2025-03-27.

[48] Cédric Teyton, Jean-Rémy Falleri, Marc Palyart, and Xavier Blanc. 2014. A study of library migrations in Java. *J. Softw. Evol. Process* 26, 11 (Nov. 2014), 1030–1052. doi:10.1002/smr.1660

[49] Rijnard van Tonder. 2025. *Comby.* Retrieved July 18, 2025 from https://comby.dev/docs/overview

[50] Rijnard van Tonder and Claire Le Goues. 2019. Lightweight Multi-Language Syntax Transformation with Parser Parser Combinators. In *Proc. ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 363–378.

[51] Ganesh Venkataraman et al. 2024. Uber's Strategy to Upgrading 2M+ Spark Jobs. https://www.uber.com/blog/ubers-strategy-to-upgrading-2m-spark-jobs/. Uber Engineering Blog.

[52] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2022. Self-instruct: Aligning language models with self-generated instructions. *arXiv preprint arXiv:2212.10560* (2022).

[53] Laerte Xavier, Aline Brito, André C. Hora, and Marco Túlio Valente. 2017. Historical and impact analysis of API breaking changes: A large-scale study. In *Proc. IEEE International Conference on Software Analysis, Evolution and Reengineering*, Martin Pinzger, Gabriele Bavota, and Andrian Marcus (Eds.). 138–147.

[54] Shengzhe Xu, Ziqi Dong, and Na Meng. 2019. Meditor: inference and application of API migration edits. In *Proc. of the International Conference on Program Comprehension*, Yann-Gaël Guéhéneuc, Foutse Khomh, and Federica Sarro (Eds.). IEEE / ACM, 335–346.

[55] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. https://openreview.net/forum?id=WE_vluYUL-X

[56] Ahmed Zaki and Cristian Cadar. 2025. Understanding API Usage and Testing: An Empirical Study of C Libraries. In *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering, EASE 2025, Istanbul, Turkey, June 17-20, 2025*, Muhammad Ali Babar, Ayse Tosun, Stefan Wagner, and Viktoria Stray (Eds.). ACM, 557–567. doi:10.1145/3756681.3756942

[57] Celal Ziftci, Stoyan Nikolov, Anna Sjövall, Bo Kim, Daniele Codecasa, and Max Kim. 2025. Migrating Code At Scale With LLMs At Google. *arXiv preprint arXiv:2504.09691* (2025).